What I Built

I built Intellisys, a multi-agent research platform that takes a plain-language research question and turns it into a cited, structured report. Under the hood it's five agents — Supervisor, Research, Analyst, Critic, and Writer — wired together with LangGraph, backed by FastAPI and SQLite, with a plain HTML/CSS/JS dashboard that lets me watch the whole thing run live instead of staring at a terminal.

Biggest Challenge

The hardest problem wasn't getting five agents to talk to each other — LangGraph made the control flow (including the Critic's revision loop) pretty manageable once I understood how conditional edges worked. The real challenge was quality: my early reports all read the same regardless of topic. A technical survey and a business decision came out with identical section headings, and worse, the Analyst was forcing a comparison table onto topics that had nothing to compare. That's what I'd call the 'generic report' problem, and fixing it took more iteration than the orchestration logic did.

Debugging Experience

One bug stands out: early on, a missing API key would get retried three times by tenacity and then surface as a RetryError that told me nothing useful. I'd stare at a wall of retry traceback instead of a simple 'you forgot to set OPENROUTER_API_KEY.' The fix was splitting complete() into two methods — one that checks configuration and fails immediately with no retry, and a separate inner method that only retries genuine network failures. That one change made every subsequent debugging session faster, because I could finally trust that a retry meant a real network problem, not a config typo.

Architecture Decisions

I made a deliberate call early on to ban any kind of mock or fallback mode from the application code itself. If OpenRouter or Tavily isn't configured, or a call fails, the run fails loudly with a real error message — it never quietly returns fake data. That decision shaped a lot of what came after: it's why I check configuration twice (once at the API boundary, once again inside the LLM client), and it's why the References section in the final report is built directly from database rows instead of trusting the LLM to list its own sources. I didn't want to ever wonder whether a citation was real.

Lessons Learned

Prompting for structure instead of forcing a template was the single highest-leverage change I made. Telling the Writer to choose its own headings based on the topic, instead of always outputting Introduction/Body/Conclusion, immediately made reports feel like they were actually about their subject. I also learned that a hard, code-level cap (the revision limit) is worth more than a well-worded prompt — I don't have to trust the Critic to behave, because the loop physically cannot run forever regardless of what it says.

Problems Encountered

Beyond the retry/RetryError issue, I hit a routing bug where /dashboard.html 404'd after I merged the landing page into index.html and forgot to update a stale link. Small, but a good reminder that static file serving via StaticFiles(html=True) has its own quiet assumptions about what maps to what.

Improvements Made

Splitting the LLM client's config check from its retry logic, rewriting the Writer to choose adaptive structure, making the Analyst's comparison table conditional instead of forced, and moving to immediate per-result evidence writes instead of batching — that last one meant a failure in the Analyst or Critic stage no longer meant losing research I'd already paid for.

Cost Observations

I haven't yet run a systematic cost measurement across many workflows — that's flagged honestly in the Evaluation Results document rather than made up. Anecdotally, the revision loop is the biggest cost lever: a run that never triggers a revision is noticeably cheaper than one that hits the cap, which is exactly why Experiment 5 (Revision Limit Comparison) is the follow-up I most want to actually run.

Performance Observations

Parallelizing the research stage with asyncio.gather() was a clear win the moment I tried it — six sequential Tavily calls versus six concurrent ones is not subtle. Everything downstream of that (Analyst, Critic, Writer) is still one call at a time, so total run time is really governed by how many revision cycles happen, not by research.

Reflection

What I'm proudest of isn't any single agent — it's the handful of hard guarantees that don't depend on the LLM behaving well: the revision cap, the deterministic References section, and the fail-fast configuration checks. Those are the parts of this system I trust completely, precisely because they're enforced in code rather than in a prompt. Everything else — plan quality, analysis depth, writing style — is still riding on how good the underlying model is on any given call, and being honest about that distinction is probably the most useful thing I've internalized building this.